192425192

G Vishnu Madhav

EXP13:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int mod26(int x) { return ((x % 26) + 26) % 26; }

int modinv(int a) {
    int x;
    a = mod26(a);
    for (x = 1; x < 26; x++) if ((a * x) % 26 == 1) return x;
    return -1;
}

/* Known plaintext attack on a 2x2 Hill cipher.
   Columns of P are plaintext blocks, columns of C are the matching ciphertext
   blocks, so C = K * P (mod 26) and  K = C * P^-1 (mod 26). */
int main() {
    char pt[10], ct[10];
    int P[2][2], C[2][2], Pi[2][2], K[2][2];
    int det, dinv, i, j;

    printf("Enter 4 plaintext letters : ");
    scanf("%4s", pt);
    printf("Enter 4 ciphertext letters: ");
    scanf("%4s", ct);

    for (i = 0; i < 2; i++)
```

```
        for (j = 0; j < 2; j++) {
            P[i][j] = toupper(pt[2 * j + i]) - 'A';
            C[i][j] = toupper(ct[2 * j + i]) - 'A';
        }

    det = mod26(P[0][0] * P[1][1] - P[0][1] * P[1][0]);
    dinv = modinv(det);
    printf("\ndet(P) mod 26 = %d\n", det);
    if (dinv < 0) {
        printf("P is not invertible mod 26, choose different plaintext blocks.\n");
        return 0;
    }
    Pi[0][0] = mod26(dinv * P[1][1]);   Pi[0][1] = mod26(-dinv * P[0][1]);
    Pi[1][0] = mod26(-dinv * P[1][0]);  Pi[1][1] = mod26(dinv * P[0][0]);

    for (i = 0; i < 2; i++)
        for (j = 0; j < 2; j++)
            K[i][j] = mod26(C[i][0] * Pi[0][j] + C[i][1] * Pi[1][j]);

    printf("P inverse = [%d %d; %d %d]\n", Pi[0][0], Pi[0][1], Pi[1][0], Pi[1][1]);
    printf("Recovered key K = C * P^-1 mod 26:\n");
    printf("  | %2d %2d |\n  | %2d %2d |\n", K[0][0], K[0][1], K[1][0], K[1][1]);
    return 0;
}
```

OUTPUT:

```
Enter 4 plaintext letters : HELP
Enter 4 ciphertext letters: BLDE

det(P) mod 26 = 9
P inverse = [19 19; 14 21]
Recovered key K = C * P^-1 mod 26:
  |  9  4 |
  |  5  7 |
```